

Architecture Diagram

Pendahuluan

Sistem Lost & Found IPB mengimplementasikan arsitektur keamanan informasi berlapis untuk melindungi data pengguna, keabsahan laporan barang hilang/ditemukan, serta kerahasiaan komunikasi data. Keamanan informasi pada sistem ini memprioritaskan:

A. Autentikasi dan Otorisasi:

Sistem menggunakan token akses JSON Web Token (JWT) yang diimplementasikan di atas skema standar industri OAuth2 Bearer Token.

B. Proteksi Kredensial dan Sandi:

Kata sandi pengguna sama sekali tidak disimpan dalam bentuk teks biasa (plaintext), melainkan di-hash secara aman menggunakan algoritma bcrypt berkekuatan tinggi yang dilengkapi salt dinamis per pengguna.

C. Keamanan Jalur Transmisi:

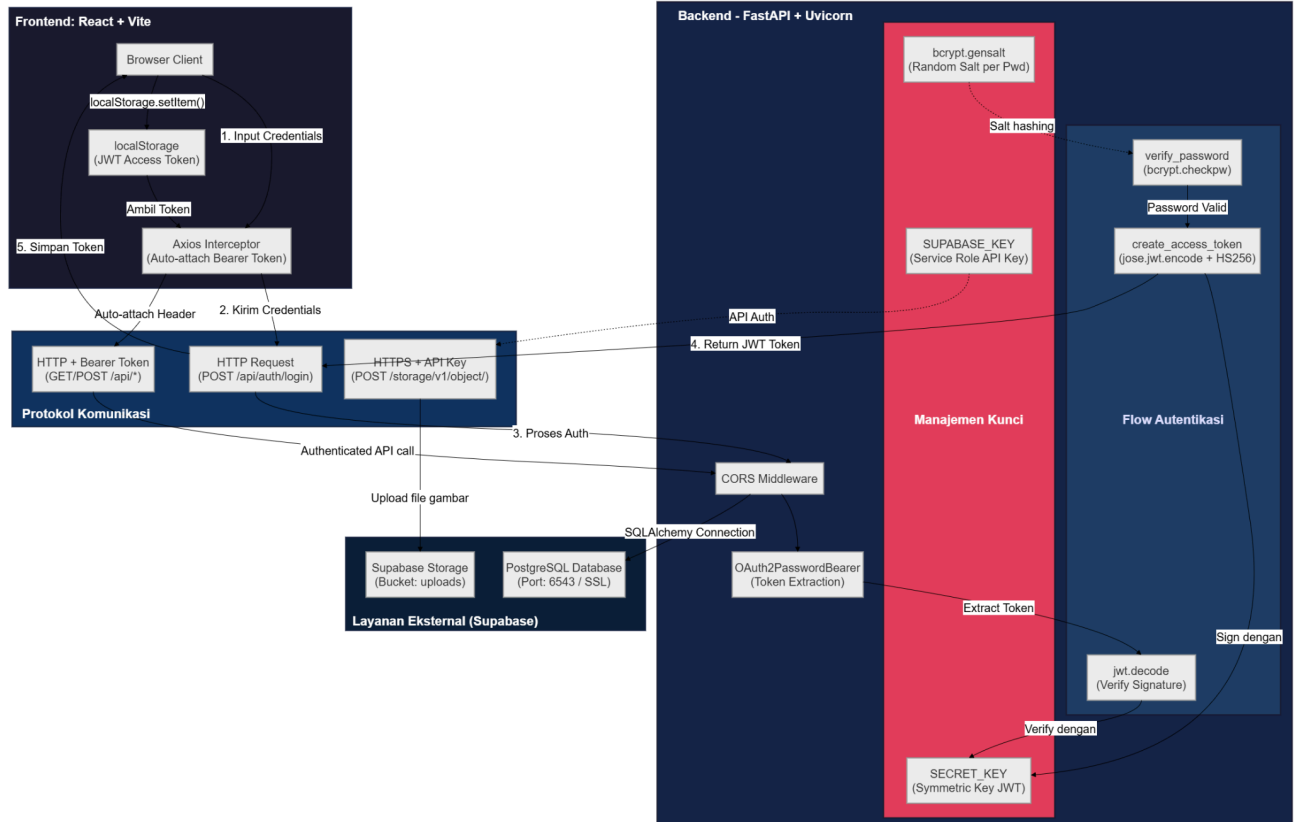
Komunikasi data ke layanan eksternal (Database PostgreSQL Supabase dan Supabase Storage) diamankan menggunakan protokol terenkripsi SSL/TLS.

D. Manajemen Kunci Rahasia:

Kunci simetris server (SECRET_KEY) dijaga kerahasiaannya dalam konfigurasi berkas .env untuk mengamankan proses validasi dan pembuatan tanda tangan digital token JWT.

Diagram Arsitektur Utama

Diagram ini memetakan interaksi statis dan komponen pelindung keamanan di seluruh lapisan sistem (Client, Jaringan, Backend, dan Layanan Eksternal).



A. Lapisan Klien (Client Layer — React SPA)

1. **Browser Client**: Aplikasi frontend berbasis React yang dijalankan oleh pengguna di browser. Sisi klien tidak dipercayakan untuk menyimpan data krusial dalam format teks biasa (plaintext).
2. **localStorage**: Area penyimpanan lokal browser yang digunakan untuk menyimpan JWT Access Token yang didapat saat login berhasil. Token ini digunakan untuk menjaga status pengguna tetap aktif.
3. **Axios Interceptor**: Modul otomatis di frontend yang mendeteksi setiap request keluar. Interceptor ini secara otomatis menyematkan header otorisasi "Authorization: Bearer [JWT_Token]" pada setiap request, sehingga meminimalisir redundansi kode dan meningkatkan keamanan komunikasi.

B. Jalur Komunikasi (Network Layer)

1. **HTTP Request POST (Login)**: Pengiriman data nama pengguna (email) dan sandi dilakukan menggunakan format payload di body request. Hal ini menyembunyikan parameter kredensial dari log server web maupun riwayat pencarian browser.
2. **HTTP Request (Protected API)**: Merupakan jalur komunikasi terproteksi yang mensyaratkan token JWT. Server akan otomatis menolak request yang tidak memuat token valid.

3. HTTPS REST Request (Supabase Storage): Pengunggahan aset gambar barang temuan dilakukan langsung dari backend ke Supabase Storage menggunakan protokol terenkripsi HTTPS demi mengamankan file biner dari penyadapan.

C. Logika Backend (FastAPI Server)

1. CORS Middleware: Konfigurasi pembatasan domain asal (origin) yang diizinkan untuk mengakses backend, berfungsi menolak request ilegal lintas asal (Cross-Origin attacks).
2. OAuth2 Header Extractor: Komponen otomatis FastAPI yang mengekstrak token Bearer dari HTTP header dan memproses validasinya.
3. Modul Autentikasi: Kumpulan fungsi backend untuk hashing password, verifikasi password, serta pembuatan dan pembedahan klaim token JWT.

D. Manajemen Kunci Kriptografi (Key Management)

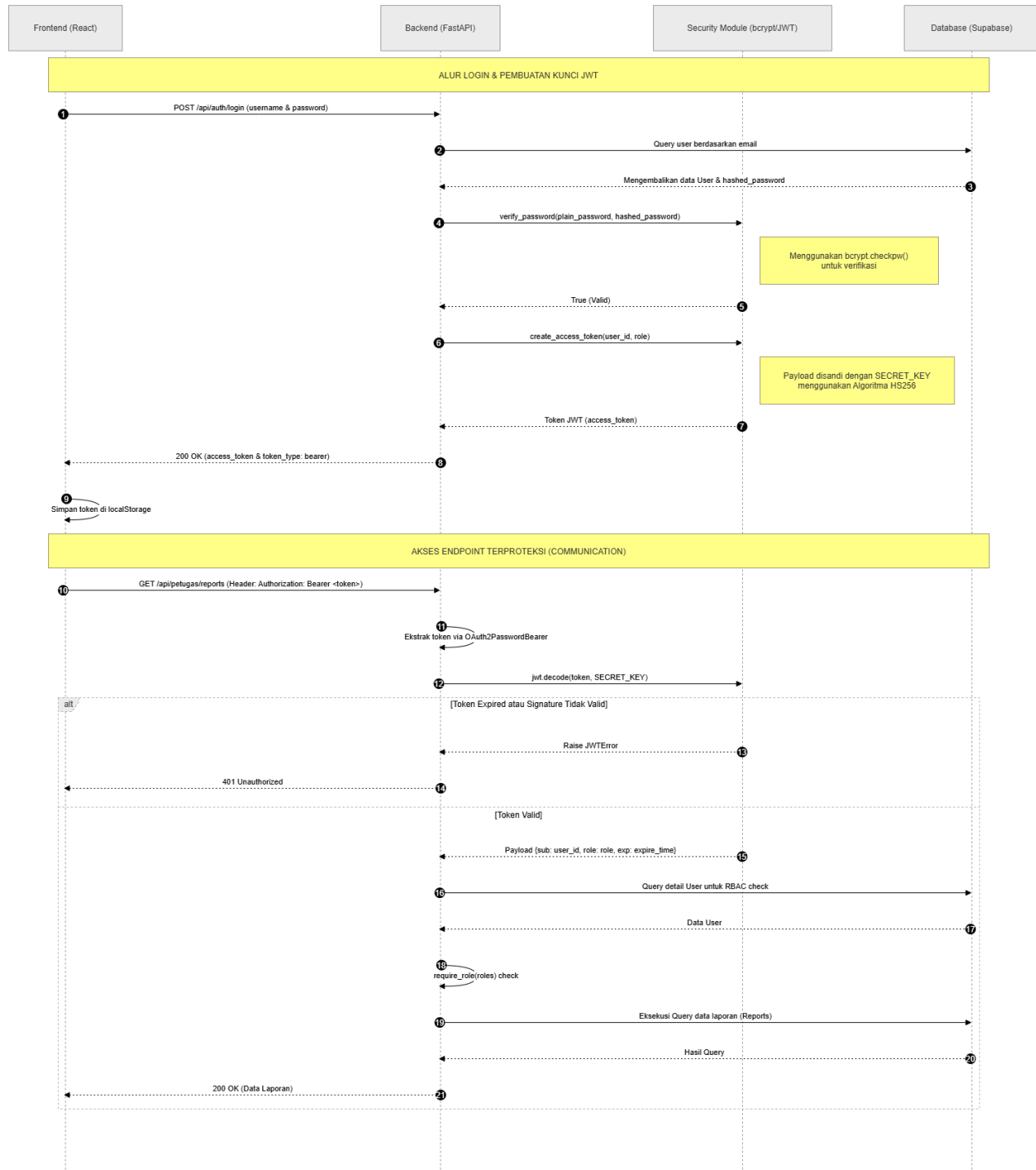
1. SECRET_KEY: Kunci simetris rahasia di sisi backend (dideklarasikan pada berkas .env). Kunci ini menjadi pondasi keamanan tanda tangan digital JWT dengan algoritma HS256. Kunci yang sama digunakan untuk proses penandatanganan (signing) dan pembuktian (verifikasi) token.
2. Bcrypt Salt: Pembangkit garam kriptografi acak berukuran 16-byte menggunakan fungsi bcrypt.gensalt(). Digunakan pada saat registrasi untuk menghasilkan hash password yang unik bagi setiap pengguna guna menepis serangan Dictionary Attack.
3. SUPABASE_KEY: Kunci rahasia tingkat tinggi (Service Role Key) milik platform Supabase yang disimpan dengan aman di server backend untuk otorisasi akses penuh ke bucket penyimpanan file.

E. Layanan Eksternal (Supabase)

1. PostgreSQL Database Connection: Pertukaran data transaksional dari backend FastAPI ke Supabase menggunakan SSL/TLS terenkripsi (Port 6543) guna menepis serangan penyadapan data (Wiretapping).
2. Supabase Storage: Penyimpanan file gambar temuan di bucket "uploads" yang terisolasi dan hanya dapat diunggah melalui request backend yang terverifikasi.

Diagram Alur Autentikasi JWT dan Manajemen Kunci

Diagram sequence ini menggambarkan alur waktu (temporal) pesan serta logika kriptografi yang dijalankan selama proses pembuatan kunci (login) dan verifikasi kunci (akses data terproteksi).



A. Fase 1: Proses Autentikasi Kredensial dan Pembuatan Kunci JWT (Login)

1. Pengguna memasukkan email dan kata sandi di form login frontend.
2. Frontend membungkus kredensial tersebut di body request dan memanggil API POST `"/api/auth/login"` ke backend FastAPI.

3. Server backend menanyakan profil akun pengguna ke database Supabase PostgreSQL berdasarkan alamat email.
4. Database mengembalikan record data pengguna yang berisi metadata profil serta sandi yang tersimpan dalam format hash (bcrypt).
5. Server mengirimkan sandi teks biasa masukan dari pengguna dan sandi hash dari database ke modul kriptografi untuk diverifikasi.
6. Modul kriptografi memproses verifikasi menggunakan fungsi `bcrypt.checkpw()`. `bcrypt` secara otomatis mengekstrak parameter salt asli yang menempel di hash database, melakukan komputasi enkripsi ulang sandi masukan dengan salt tersebut, lalu mencocokkannya. Jika cocok, status valid (True) dikembalikan.
7. Backend meminta pembuatan token akses JWT baru dengan menyisipkan identitas user ID (sub) dan tipe peran (role).
8. Modul kriptografi menyandikan klaim payload tersebut dan menandatangani dengan kunci `SECRET_KEY` menggunakan algoritma tanda tangan digital HMAC-SHA256 (HS256).
9. Token JWT yang telah ditandatangani dikirim kembali ke frontend dengan status 200 OK.
10. Frontend menyimpan token JWT tersebut di `localStorage` browser guna mengautentikasi aktivitas request berikutnya.

B. Fase 2: Akses Secure Endpoint dan Komunikasi Data

1. Pengguna memicu aksi yang membutuhkan keamanan (misalnya melihat laporan barang temuan). Frontend mengirimkan request `GET "/api/petugas/reports"` ke backend dengan menyertakan token di HTTP header `"Authorization: Bearer [JWT_Token]"`.
2. Server backend mengekstrak token menggunakan `OAuth2PasswordBearer` dan meminta modul kriptografi mendekode tanda tangan token menggunakan kunci `SECRET_KEY`.
3. Jika token telah melampaui masa kedaluwarsa 30 menit atau tanda tangannya tidak cocok (akibat modifikasi ilegal), modul kriptografi melempar pengecualian (`JWTError`). Server langsung menolak request dengan status 401 Unauthorized. Frontend menangkap error ini, menghapus token lokal, dan mengarahkan pengguna kembali ke form login.
4. Jika tanda tangan valid, modul mengembalikan payload berisi data klaim user ID (sub) dan role.
5. Server FastAPI menjalankan otorisasi tingkat lanjut (Role-Based Access Control) dengan memeriksa apakah role pengguna berhak mengakses endpoint tersebut melalui dependensi `require_role()`.
6. Jika diizinkan, server mengeksekusi query pengambilan data laporan barang ke database PostgreSQL dan mengembalikan hasilnya ke frontend dengan response status 200 OK.
7. Frontend menerima payload JSON dan merendernya secara aman pada antarmuka visual pengguna.

Parameter Keamanan Informasi

A. Kriptografi Password

- Teknologi: bcrypt
- Parameter Kunci: Cost Factor 12, Salt 16-byte otomatis per pengguna
- Lokasi Penyimpanan: PostgreSQL database (tabel users kolom hashed_password)
- Tujuan: Melindungi data kata sandi dari kebocoran fisik database dan serangan bruteforce.

B. Tanda Tangan Token

- Teknologi: JSON Web Token (JWT)
- Parameter Kunci: Algoritma HMAC-SHA256 (HS256) menggunakan SECRET_KEY server
- Lokasi Penyimpanan: Client localStorage (sementara) dan .env (SECRET_KEY di server)
- Tujuan: Menjamin integritas dan keaslian token selama komunikasi berlangsung.

C. Batas Waktu Token

- Parameter: 30 Menit (Konfigurasi ACCESS_TOKEN_EXPIRE_MINUTES)
- Tujuan: Mengurangi risiko pembajakan token akses (Session Hijacking) jika komputer klien ditinggal dalam keadaan aktif.

D. Protokol Transmisi DB

- Teknologi: PostgreSQL Wire Protocol + SSL terenkripsi (Port 6543)
- Driver Koneksi: psycopg (SQLAlchemy ORM)
- Tujuan: Menghindari serangan Man-in-the-Middle (penyadapan data di tengah jalur jaringan).

E. Otentikasi Berkas

- Teknologi: HTTPS REST API + Supabase Service Role Key
- Filter Keamanan: Validasi Tipe Konten Gambar (image/*) di backend
- Tujuan: Menjamin unggahan aset gambar hanya dilakukan secara sah dari server terverifikasi dan menolak unggahan skrip berbahaya.